Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy.

Operate as one coordinated team. Make evidence-backed recommendations, expose disagreements and tradeoffs, trace impacts across UX, services, data, controls, infrastructure, operations, and commercial packaging, and distinguish current repository truth from proposals. Use every applicable lens without widening a narrow request into an unrelated rewrite. Mark immaterial lenses `not applicable` with one short reason. Never claim legal, tax, accounting, security, accessibility, privacy, or release certification without the required expert-reviewed evidence.

Workspace: E:\ohada saas\Focused projects\stoquify

Mission: Determine which combination of architectural approach, programming practices, coding standards, and engineering governance best fits Stoquify’s actual codebase.

Explain how this approach would make the system easier to understand, change, test, operate, secure, and scale. Recommend a coherent approach, justify why it fits, and provide a practical adoption plan.

This is an assessment and recommendation task. Produce a report and illustrative examples; do not modify application code, dependencies, schema, or infrastructure.

Include specialist review of POS/inventory/offline synchronization; purchasing/AP; HRIS/payroll; payments/reconciliation; accounting close; audit evidence and records; analytics; AI agents; external integrations; and rollout/support readiness. Give each review role an evidence-backed finding or a short “not applicable” explanation.

1. Establish repository truth

Read applicable AGENTS.md instructions and inspect:

- package.json, lockfiles, TypeScript configuration, lint/format configuration, CI workflows, and testing tools.
- Relevant architecture and engineering documentation.
- Relevant what-next/ and innovation/ reports.
- Existing graphify-out/ graphs and reports for components, actions, routes, hooks, and types.
- app/, actions/, services/, components/, hooks/, lib/, config/, prisma/, scripts/, and focused tests, where present.

Use graphs to identify relationships and dependencies, then validate consequential claims against current source code. Reference relevant graph nodes or communities when useful; report stale or missing graph evidence.

Trace at least three representative workflows:

- A financially or operationally sensitive write.

- A tenant-scoped read that supplies a user interface.
- An asynchronous, provider, webhook, or offline synchronization flow, if one exists.

For each, follow the path through entry point, authentication, authorization, validation, business logic, persistence, side effects, error handling, and tests.

Separate observed facts, inferred risks, proposals, and unknowns. Cite file paths and line numbers for significant findings. State the sampling limits.

2. Define what “professional and enterprise grade” means

Distinguish:

- Code style: naming, formatting, file organization, and consistency.
- Programming practices: types, composition, functions/classes, validation, and error handling.
- Architecture: domain boundaries, dependency direction, contracts, and ownership.
- Engineering governance: tests, reviews, CI enforcement, documentation, and ownership.
- Operational quality: observability, resilience, deployment, recovery, performance, and cost.

Define measurable quality goals for Stoquify. Distinguish scaling traffic and data from scaling teams, modules, and change volume.

Identify which existing practices should be preserved.

3. Evaluate approaches and recommend a combination

Evaluate plausible options, including:

- Improving the existing structure with stronger conventions and enforcement.
- A modular monolith organized around business capabilities.
- Feature or vertical-slice organization within domain boundaries.
- Selective domain-driven design.
- Ports-and-adapters or clean-architecture principles at boundaries where they solve an observed problem.
- Functional core / imperative shell for suitable business logic.
- Selective CQRS, events, or independently deployed services where evidence supports their cost.

These approaches operate at different levels and may be combined. Explain how the recommended combination fits together.

Do not select a pattern because it is fashionable. Do not assume that enterprise quality requires microservices, a framework migration, universal repositories, or additional abstraction layers.

Compare credible alternatives using:

- Fit with current code and domain complexity.
- Understandability and onboarding.
- Testability and business correctness.
- Security and tenant isolation.
- Change isolation and team ownership.
- Performance and operational complexity.
- Migration effort and ongoing maintenance cost.

Lead with a clear recommendation. Explain why it fits Stoquify, where it should apply, where it should not apply, and what evidence would justify revisiting it.

4. Define practical engineering standards

Propose concise, enforceable standards for:

- Domain/module ownership and allowed dependency direction.
- Placement of business rules, application orchestration, and persistence.
- Thin routes/actions and server-owned business truth.
- Service-owned read models and stable UI contracts.
- TypeScript strictness, runtime validation, and contract typing.
- Naming, file organization, public module interfaces, and abstraction thresholds.
- Appropriate use of functions, classes, composition, and shared utilities.
- Tenant isolation, RBAC, module entitlement, and sensitive-data handling.
- Transactions, monetary precision, concurrency, idempotency, and audit history.
- External integrations, retries, timeouts, events, and recovery.
- Frontend state ownership, accessibility, localization, and recoverable UI states.
- Tests, observability, documentation, architecture decisions, and CI enforcement.

For every standard, explain:

- The problem it solves.
- The repository evidence supporting it.
- Its cost or tradeoff.
- How adoption would be verified.

Keep statutory and country-specific rules behind explicit, versioned boundaries with reviewed provenance.

5. Make the recommendation concrete

Provide:

- A proposed directory/module structure adapted to the existing repository.
- A simple diagram of permitted dependencies.
- A walkthrough mapping one existing workflow to the proposed structure.
- Two short before/after examples grounded in inspected code: one business operation and one frontend/service boundary.
- Examples of good existing code worth preserving.

Label examples as illustrative and unexecuted. Explain the practical benefit of each change. Avoid examples that merely add files, wrappers, or interfaces without improving a demonstrated problem.

6. Produce an incremental adoption plan

Prioritize work by observed risk, user impact, maintenance burden, and dependency order.

Separate:

- Immediate conventions and low-risk improvements.
- One bounded pilot in a real workflow.

- Structural changes justified by pilot evidence.
- Conditional changes to defer until a measurable trigger occurs.

For each stage, identify affected areas, prerequisites, effort assumptions, verification, rollback strategy, and completion criteria.

Recommend where boundaries can be enforced automatically. Explain how existing violations can be reduced without blocking unrelated work.

Avoid a broad rewrite or mass file movement.

7. Verification and risk controls

Start with read-only baseline discovery:

- git status --short
- git diff --stat
- Inspect package.json and the lockfile to identify the package manager and available scripts.

Inspect scripts before running them. Run only existing, relevant checks that do not mutate source, shared data, or external environments. Record exact commands, results, and limitations.

Do not invent command names or claim checks passed without running them. Separate existing failures from risks inferred during review.

Preserve unrelated work and do not fix unrelated lint warnings. Do not install dependencies, run migrations, reseed databases, deploy, or alter security and financial controls.

Do not infer production capacity from code structure alone. Where runtime evidence is missing, propose specific measurements or benchmarks.

8. Expected artifacts

Save: what-next/stoquify-engineering-approach-review.md

Include:

- A concise recommendation and rationale.
- Current-state findings with source evidence.
- An alternatives comparison with tradeoffs.
- Proposed architecture and engineering standards.
- Illustrative examples.
- A prioritized migration plan.
- Verification results, unknowns, and unresolved decisions.
- A coverage statement for the review roles.

Keep the final chat response focused on the recommended approach, the strongest supporting evidence, and the first practical step.

9. Success criteria and non-goals

The review is complete when:

- The recommendation is specific to inspected Stoquify code.

- Architectural choices and coding conventions are clearly distinguished.
- Benefits, costs, alternatives, and adoption conditions are explained.
- The plan preserves working behavior and critical business controls.
- The pilot has a bounded scope and measurable completion criteria.
- Every significant claim is supported or explicitly marked uncertain.

Non-goals: A complete rewrite, cosmetic cleanup, speculative infrastructure, new product features, and unsupported enterprise-readiness certification.

End with two optional follow-up prompts:

- Implement the recommended bounded pilot.
- Evaluate the pilot and introduce the justified engineering checks.